

Transformers without Normalization

Jiachen Zhu^{1,2}, Xinlei Chen¹, Kaiming He³, Yann LeCun^{1,2}, Zhuang Liu^{1,4,†}

¹FAIR, Meta, ²New York University, ³MIT, ⁴Princeton University

[†]Project lead

Normalization layers are ubiquitous in modern neural networks and have long been considered essential. This work demonstrates that Transformers without normalization can achieve the same or better performance using a remarkably simple technique. We introduce Dynamic Tanh (DyT), an element-wise operation $\text{DyT}(\mathbf{x}) = \tanh(\alpha\mathbf{x})$, as a drop-in replacement for normalization layers in Transformers. DyT is inspired by the observation that layer normalization in Transformers often produces tanh-like, S -shaped input-output mappings. By incorporating DyT, Transformers without normalization can match or exceed the performance of their normalized counterparts, mostly without hyperparameter tuning. We validate the effectiveness of Transformers with DyT across diverse settings, ranging from recognition to generation, supervised to self-supervised learning, and computer vision to language models. These findings challenge the conventional understanding that normalization layers are indispensable in modern neural networks, and offer new insights into their role in deep networks.

Date: June 17, 2025

Project page and code: jiachenzhu.github.io/DyT

Correspondence: jiachen.zhu@nyu.edu, zhuangl@princeton.edu



1 Introduction

Over the past decade, normalization layers have solidified their positions as one of the most fundamental components of modern neural networks. It all traces back to the invention of batch normalization in 2015 (Ioffe and Szegedy, 2015), which enabled drastically faster and better convergence in visual recognition models and quickly gained momentum in the following years. Since then, many variants of normalization layers have been proposed for different network architectures or domains (Ba et al., 2016; Ulyanov et al., 2016; Wu and He, 2018; Zhang and Sennrich, 2019). Today, virtually all modern networks use normalization layers, with layer normalization (Layer Norm, or LN) (Ba et al., 2016) being one of the most popular, particularly in the dominant Transformer architecture (Vaswani et al., 2017; Dosovitskiy et al., 2020).

The widespread adoption of normalization layers is largely driven by their empirical benefits in optimization (Santurkar et al., 2018; Bjorck et al., 2018). In addition to achieving better results, they help accelerate and stabilize convergence. As neural networks become wider and deeper, this necessity becomes ever more critical (Brock et al., 2021a; Huang et al., 2023). Consequently, normalization layers are widely regarded as crucial, if not indispensable, for the effective training of deep networks. This belief is subtly evidenced by the fact that, in recent years, novel architectures often seek to replace attention or convolution layers (Tolstikhin et al., 2021; Gu and Dao, 2023; Sun et al., 2024; Feng et al., 2024), but almost always retain the normalization layers.

This paper challenges this belief by introducing a simple alternative to normalization layers in Transformers. Our exploration starts with the observation that LN layers map their inputs to outputs with tanh-like, S -shaped curves, scaling the input activations while squashing the extreme values. Inspired by this insight, we propose an element-wise operation termed Dynamic Tanh (DyT), defined as: $\text{DyT}(\mathbf{x}) = \tanh(\alpha\mathbf{x})$, where α is a learnable parameter. This operation aims to emulate the behavior of LN by learning an appropriate scaling factor through α and squashing extreme values via the bounded tanh function. Notably, unlike normalization layers, it achieves both effects without the need to compute activation statistics.

Employing DyT is straightforward, as shown in Figure 1: we directly replace existing normalization layers with DyT in architectures such as vision and language Transformers. We empirically demonstrate that models with DyT can train stably and achieve high final performance across a wide range of settings. It often does not

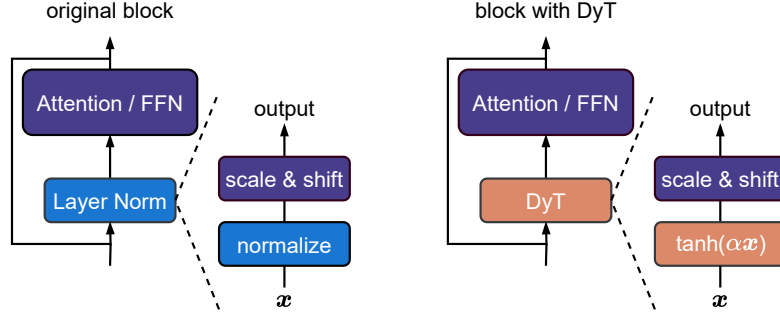


Figure 1 *Left:* original Transformer block. *Right:* block with our proposed Dynamic Tanh (DyT) layer. DyT is a straightforward replacement for commonly used Layer Norm (Ba et al., 2016) (in some cases RMSNorm (Zhang and Sennrich, 2019)) layers. Transformers with DyT match or exceed the performance of their normalized counterparts.

require tuning the training hyperparameters on the original architecture. Our work challenges the notion that normalization layers are indispensable for training modern neural networks and provides empirical insights into the properties of normalization layers.

2 Background: Normalization Layers

We begin by reviewing the normalization layers. Most normalization layers share a common formulation. Given an input x with shape (B, T, C) , where B is the batch size, T is the number of tokens, and C is the embedding dimension per token, the output is generally computed as:

$$\text{normalization}(x) = \gamma * \left(\frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \right) + \beta \quad (1)$$

where ϵ is a small constant, and γ and β are learnable vector parameters of shape $(C,)$. They are “scaling” and “shifting” affine parameters that allow the output to be in any range. The terms μ and σ^2 denote the mean and variance of the input. Different methods mainly differ in how these two statistics are computed. This results in μ and σ^2 having different dimensions, each with broadcasting applied during computation.

Batch normalization (BN) (Ioffe and Szegedy, 2015) is the first modern normalization layer, and it has been primarily used in ConvNet models (Szegedy et al., 2016; He et al., 2016; Xie et al., 2017). Its introduction represents a major milestone in deep learning architecture designs. BN computes the mean and variance across both the batch and token dimensions, specifically: $\mu_k = \frac{1}{BT} \sum_{i,j} x_{ijk}$ and $\sigma_k^2 = \frac{1}{BT} \sum_{i,j} (x_{ijk} - \mu_k)^2$. Other normalization layers popular in ConvNets, such as group normalization (Wu and He, 2018) and instance normalization (Ulyanov et al., 2016), were initially proposed for specialized tasks such as object detection and image stylization. They share the same overall formulation but differ in the axes and ranges over which the statistics are computed.

Layer normalization (LN) (Ba et al., 2016) and root mean square normalization (RMSNorm) (Zhang and Sennrich, 2019) are the major two types of normalization layers used in Transformer architectures. LN computes these statistics independently for each token in each sample, where $\mu_{ij} = \frac{1}{C} \sum_k x_{ijk}$ and $\sigma_{ij}^2 = \frac{1}{C} \sum_k (x_{ijk} - \mu_{ij})^2$. RMSNorm (Zhang and Sennrich, 2019) simplifies LN by removing the mean-centering step and normalizing the input with $\mu_{ij} = 0$ and $\sigma_{ij}^2 = \frac{1}{C} \sum_k x_{ijk}^2$. Today, most modern neural networks use LN due to its simplicity and universality. Recently, RMSNorm has gained popularity, particularly in language models like T5 (Raffel et al., 2020), LLaMA (Touvron et al., 2023a,b; Dubey et al., 2024), Mistral (Jiang et al., 2023), Qwen (Bai et al., 2023; Yang et al., 2024), InternLM (Zhang et al., 2024; Cai et al., 2024) and DeepSeek (Liu et al., 2024; Guo et al., 2025). The Transformers we examine in this work all use LN, except that LLaMA uses RMSNorm.

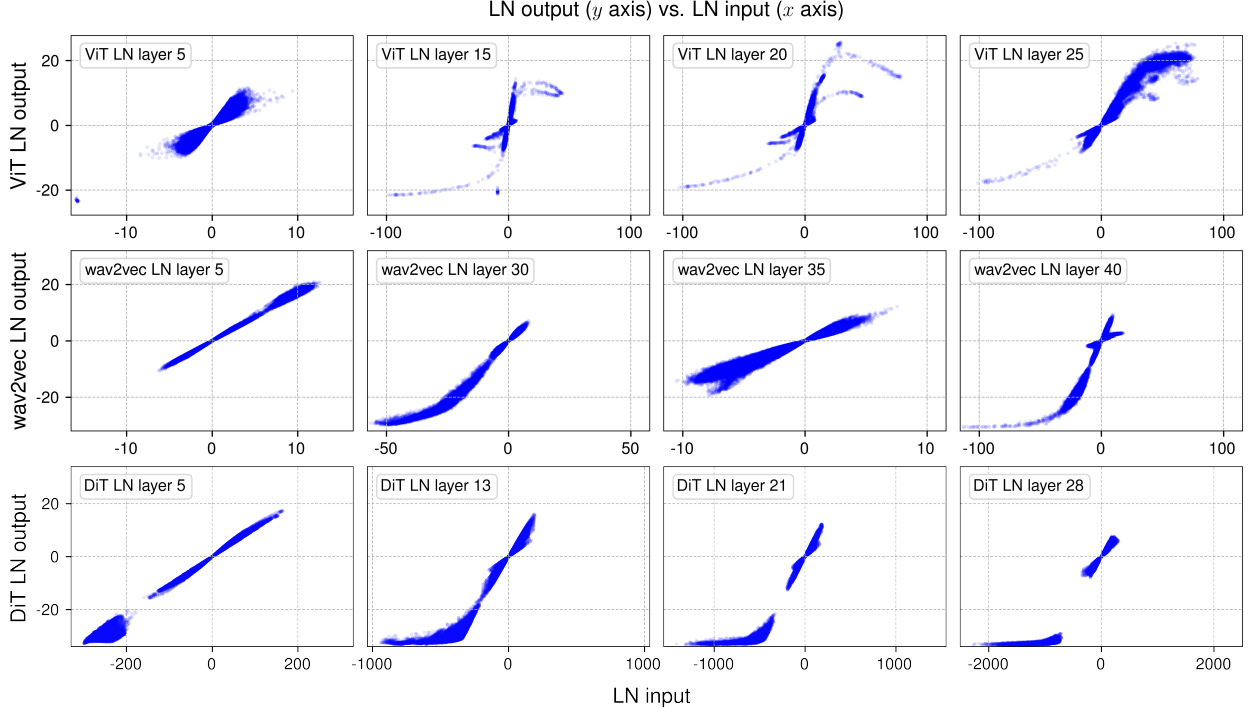


Figure 2 Output vs. input of selected layer normalization (LN) layers in Vision Transformer (ViT) (Dosovitskiy et al., 2020), wav2vec 2.0 (a Transformer model for speech) (Baevski et al., 2020), and Diffusion Transformer (DiT) (Peebles and Xie, 2023). We sample a mini-batch of samples and plot the input / output values of four LN layers in each model. The outputs are before the affine transformation in LN. The S -shaped curves highly resemble that of a tanh function (see Figure 3). The more linear shapes in earlier layers can also be captured by the center part of a tanh curve. This motivates us to propose Dynamic Tanh (DyT) as a replacement, with a learnable scaler α to account for different scales on the x axis.

3 What Do Normalization Layers Do?

Analysis setup. We first empirically study the behaviors of normalization layers in trained networks. For this analysis, we take a Vision Transformer model (ViT-B) (Dosovitskiy et al., 2020) trained on ImageNet-1K (Deng et al., 2009), a wav2vec 2.0 Large Transformer model (Baevski et al., 2020) trained on LibriSpeech (Panayotov et al., 2015), and a Diffusion Transformer (DiT-XL) (Peebles and Xie, 2023) trained on ImageNet-1K. In all cases, LN is applied in every Transformer block and before the final linear projection.

For all three trained networks, we sample a mini-batch of samples and do a forward pass through the network. We then measure the input and output for the normalization layers, i.e., tensors immediately before and after the normalization operation, before the learnable affine transformation. Since LN preserves the dimensions of the input tensor, we can establish a one-to-one correspondence between the input and output tensor elements, allowing for a direct visualization of their relationship. We plot the resulting mappings in Figure 2.

Tanh-like mappings with layer normalization. For all three models, in earlier LN layers (1st column of Figure 2), we find this input-output relationship to be mostly linear, resembling a straight line in an x - y plot. However, the deeper LN layers are places where we make more intriguing observations.

A striking observation from these deeper layers is that most of these curves’ shapes highly resemble full or partial S -shaped curves represented by a tanh function (see Figure 3). One might expect LN layers to linearly transform the input tensor, as subtracting the mean and dividing by standard deviation are

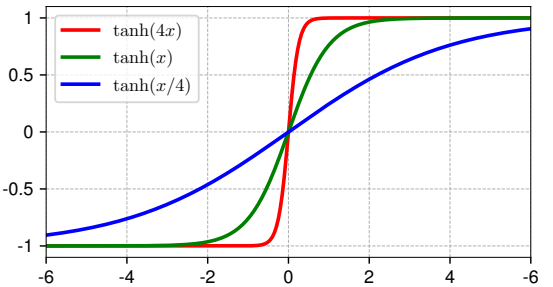


Figure 3 $\tanh(\alpha x)$ with three different α values.